TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

# YAMS: Managing JCNR with Model Context Protocol

An HPE Juniper Networking Techpost

**Lavanya Kumar Ambatipudi**

**2025-07-24**

# Contents

**1 YAMS: Managing JCNR with Model Context Protocol 2**
1.1 Introduction . . . 2
1.1.1 What is JCNR? . . . 2
1.1.2 What is YAMS? . . . 2
1.1.3 Core YAMS Architecture . . . 2
1.1.4 Why YAMS for JCNR? . . . 3
1.2 JCNR-Specific Tools in YAMS . . . 3
1.2.1 1 - cRPD Control Plane Tools . . . 3
1.2.2 2 - DPDK Data Plane Tools . . . 3
1.2.3 3 - Contrail Agent Tools . . . 4
1.2.4 4 - Comprehensive JCNR Analysis . . . 4
1.2.5 5 - Advanced Diagnostic Tools . . . 5
1.3 JCNR Network Topology . . . 5
1.3.1 Topology Overview . . . 6
1.3.2 Sample HTTP API Outputs from JCNR3 Cluster . . . 7
1.4 Comprehensive Use Case: Multi-Cluster JCNR Network Analysis . . . 9
1.4.1 Scenario: Production Network Health Assessment . . . 9
1.4.2 Step 1: BGP Summary Across All Clusters . . . 9
1.4.3 Step 2: OSPF Summary Across All Clusters . . . 10
1.4.4 Step 3: Interface Statistics from All JCNR Clusters . . . 10
1.4.5 Step 4: JCNR Summary from All Clusters . . . 11
1.4.6 Step 5: Specific Route Analysis . . . 11
1.4.7 Use Case Summary . . . 12
1.5 Installation and Setup . . . 13
1.5.1 Prerequisites . . . 13
1.5.2 Quick Start with Docker . . . 13
1.5.3 Cluster Configuration . . . 13
1.5.4 Command Configuration . . . 14
1.5.5 VS Code Integration . . . 16
1.6 Benefits and Impact . . . 16
1.6.1 Operational Benefits . . . 16
1.6.2 Technical Advantages . . . 16
1.7 Getting Started . . . 16
1.8 Conclusion . . . 17
1.9 Useful links . . . 17
1.10 Glossary . . . 17
1.11 Acknowledgments . . . 18

# 1 YAMS: Managing JCNR with Model Context Protocol

**Lavanya Kumar Ambatipudi - 07/24/2025**

YAMS (Yet Another MCP Server) is a specialized Model Context Protocol server designed to address the operational complexities of managing JCNR deployments at scale. Built specifically for network engineers, testing engineers and DevOps teams, YAMS provides a unified management interface that abstracts the complexity of multi-cluster JCNR operations.

This article was originally posted on: https://github.com/Juniper/yams/blob/main/YAMS_JCNR_Article.md#comprehensive-use-case-multi-cluster-jcnr-network-analysis

## 1.1 Introduction

### 1.1.1 What is JCNR?

Juniper Cloud-Native Router (JCNR) is Juniper's next-generation containerized routing solution that transforms traditional networking by bringing enterprise-grade routing capabilities directly into Kubernetes environments. JCNR represents a fundamental shift from hardware-centric to software-defined networking, enabling scalable, cloud-native network functions.

YAMS (Yet Another MCP Server) is a specialized Model Context Protocol server designed to address the operational complexities of managing JCNR deployments at scale. Built specifically for network engineers, testing engineers, and DevOps teams, YAMS provides a unified management interface that abstracts the complexity of multi-cluster JCNR operations.

### 1.1.2 What is YAMS?

YAMS is an HTTP-based MCP server that integrates deeply with Kubernetes clusters and JCNR components. It leverages the Model Context Protocol to provide intelligent, context-aware interactions with the JCNR infrastructure through modern development tools like VS Code Copilot Chat and other MCP-compatible desktops.

### 1.1.3 Core YAMS Architecture

- **HTTP-based MCP Server**: RESTful API design built on FastAPI framework for high performance and scalability
- **Multi-cluster Orchestration**: Seamless management of multiple Kubernetes environments with unified authentication
- **SSH Tunnel Integration**: Secure access to private cloud and on-premises JCNR deployments
- **JCNR-Native Tools**: Purpose-built commands that understand JCNR's three-tier architecture
- **VS Code Integration**: Native support for Copilot Chat, enabling natural language network operations
- **Real-time Data Access**: Direct integration with JCNR's Sandesh HTTP APIs for live operational data

#### 1.1.4 Why YAMS for JCNR?

Traditional JCNR management requires network engineers to:

- Connect to each cluster individually using different kubeconfig files
- Execute commands across multiple namespaces (contrail, jcnr) and pod types
- Correlate data between DPDK datapath, routing protocols, and agent components
- Manually aggregate information for cross-cluster analysis
- Switch between different command syntaxes (kubectl, Junos CLI, DPDK commands)

YAMS transforms this workflow into unified operations that work across all clusters simultaneously.

### 1.2 JCNR-Specific Tools in YAMS

YAMS provides specialized tools designed specifically for JCNR's three-tier architecture, enabling comprehensive management of control plane, data plane, and agent components across multiple clusters.

#### 1.2.1 1 - cRPD Control Plane Tools

Execute Junos CLI commands across all cRPD (Containerized Routing Protocol Daemon) instances in the jcnr namespace.

**Key Features**:

- Automatically prepends **cli -c** for proper Junos CLI execution
- Supports all standard Junos commands (show, configure, monitor)
- Executes across all cRPD and cSRX pods simultaneously
- Provides unified output from multiple clusters

**Common Use Cases**:

```
# BGP protocol analysis
"Show BGP summary across all clusters"
"Display BGP neighbor status and sessions"
"Get BGP route information for all protocols"

# OSPF protocol analysis
"Show OSPF neighbor adjacencies"
"Display OSPF database information"
"Get OSPF routes from all clusters"

# Interface and routing analysis
"Show interface status in terse format"
"Display routing table summary"
"Get protocol configuration from all routers"
```

#### 1.2.2 2 - DPDK Data Plane Tools

Execute commands in DPDK pods (vrdpdk) across the contrail namespace for data plane analysis.

**Key Features**:

- Targets all vrouter-nodes-vrdpdk pods across clusters
- Provides high-performance packet processing insights

- Supports all vRouter command-line tools
- Delivers unified datapath visibility

**Common Use Cases**:

```
# Interface and next-hop analysis
"List all interfaces across clusters"
"Show next-hop table information"

# Routing table analysis
"Dump IPv4 routing table for VRF 0"
"Get specific route lookup in datapath"

# Flow and performance analysis
"Display active flow information"
"Show MPLS label assignments"
```

#### 1.2.3 3 - Contrail Agent Tools

JCNR runs an agent module responsible for communicating with cRPD and programming JCNR vrouter data path. Agent provides an HTTP interface called introspect. This is a rich HTTP API interface that gives a comprehensive view of the Agent and its internal data.

**HTTP API Integration for Real-time Data**:

The Contrail Agent provides rich HTTP API endpoints that YAMS integrates with to provide formatted, real-time operational data.

**Common Use Cases**:

```
# Next-hop and routing information
"Get next-hop list from HTTP API"
"Show VRF table information from HTTP API"
```

**Benefits of HTTP API Integration**:

- **Real-time Data**: Live operational statistics without pod restart requirements
- **Structured Output**: Formatted tables for easy analysis and correlation
- **Rich Metadata**: Additional context like VRF mappings, peer information, and state details
- **Performance Insights**: Detailed packet/byte counters and error statistics
- **Policy Visibility**: Security group and ACL information for troubleshooting
- **Unified Format**: Consistent data presentation across all clusters

#### 1.2.4 4 - Comprehensive JCNR Analysis

Provides complete JCNR datapath and control plane analysis by combining data from all three tiers.

**Key Features**:

- Executes predefined command sets from configurable JSON files
- Fetches real-time data from Sandesh HTTP APIs
- Combines DPDK, Agent, and cRPD outputs in unified reports
- Supports both single-cluster and multi-cluster analysis

**Components Analyzed**:

- **DPDK Commands**: nh –list, vif –list, rt –dump, flow -l, mpls –dump
- **HTTP API Data**: Next-hop tables, VRF lists, route tables, interface statistics
- **Junos CLI**: Interface status, routing tables, protocol summaries, BGP/OSPF status

#### 1.2.5 5 - Advanced Diagnostic Tools

*Note: While these tools are showcased with JCNR deployments, they can be applied to any Kubernetes pods and workloads across your clusters for comprehensive diagnostics and monitoring.*

**pod_command_and_summary**

Execute predefined command sets on any JCNR pod with execution statistics.

**Key Features**:

- Commands loaded from configurable JSON files
- Execution summary with success rates and timing
- Supports custom command lists per cluster
- Targets specific pods for detailed analysis

**analyze_logs**

Intelligent log analysis across JCNR components.

**Key Features**:

- Searches for error patterns in DPDK, agent, and cRPD logs
- Configurable regex patterns and time windows
- Multi-node log aggregation
- Customizable output limits

**check_core_files**

Automated detection of crash dumps and core files.

**Key Features**:

- Searches common core dump locations
- Age-based filtering for recent crashes

### 1.3 JCNR Network Topology

The following diagram illustrates the interconnected JCNR cluster topology used in our multi-cluster deployment examples:

```
JCNR SR-MPLS Multi-Cluster Network Topology
==========================================
      (SR-MPLS Ring Architecture)

             +-------------+
             | JCNR3 |
             |(SR Control) |
             |jcnr3.demolab|
             +-----+---+---+
                   | |
           Physical| |Physical
              Link| |Link
```

```
                  | |
       +--------------+ +--------------+
       | |
       v v
 +-------------+ +-------------+
 | JCNR2 | | JCNR6 |
 |(SR Endpoint) | |(SR Transit) |
 |jcnr2.demolab | |jcnr6.demolab|
 +-------------+ +-------------+
       | |
 Physical| |Physical
    Link| |Link
       | |
       +--------------+ +--------------+
                  | |
                  v v
              +-------------+
              | JCNR4 |
              |(SR Control) |
              |jcnr4.demolab|
              +-------------+

SR-MPLS Ring Connections (Clockwise):
JCNR2 -- JCNR3 -- JCNR6 -- JCNR4 -- JCNR2

Physical Links:
* JCNR2 <-> JCNR3: 192.168.133.0/24
* JCNR3 <-> JCNR6: 192.168.144.0/24
* JCNR6 <-> JCNR4: 192.168.155.0/24
* JCNR4 <-> JCNR2: 192.168.200.0/24

SR-MPLS Control Plane:
JCNR3 ~~~~~~~~~~~~~~~~ JCNR4
     (SR-MPLS path via ring)

Legend:
===== Physical Links (Ring topology)

Network Details:
* JCNR2 <-> JCNR3: Direct physical link (192.168.133.0/24), OSPF adjacency, SR prefix SID
    distribution
* JCNR3 <-> JCNR6: Direct physical link (192.168.144.0/24), OSPF adjacency, SR prefix SID
    distribution
* JCNR6 <-> JCNR4: Direct physical link (192.168.155.0/24), OSPF adjacency, SR prefix SID
    distribution
* JCNR4 <-> JCNR2: Direct physical link (192.168.200.0/24), OSPF adjacency, SR prefix SID
    distribution
* JCNR3 <-> JCNR4: SR-MPLS tunnels with label stacking for L3VPN services

Physical Ring Topology:
JCNR2 -- JCNR3 -- JCNR6 -- JCNR4 -- JCNR2 (complete ring)

Cluster Roles:
* JCNR3: Control node, SR-MPLS ingress/egress
* JCNR4: Control node, SR-MPLS ingress/egress (reachable via ring)
* JCNR6: Transit node, SR-MPLS forwarding
* JCNR2: Edge node, SR-MPLS customer connections
```

### 1.3.1 Topology Overview

- **Ring Topology**: All four clusters form a logical ring for redundancy
- **SR-MPLS**: Segment Routing MPLS for simplified forwarding and traffic engineering
- **OSPF Backbone**: All clusters participate in OSPF Area 0 for IGP connectivity and SR label distribution
- **SR-MPLS L3VPN**: VPN services using Segment Routing labels for efficient forwarding
- **Load Distribution**: Traffic can flow in both directions around the ring using SR paths

**Use Case Context**:

This topology represents a typical SR-MPLS JCNR deployment where:

- Multiple clusters provide distribution into multiple K8S clusters

- OSPF distributes prefix SIDs and adjacency SIDs for Segment Routing
- SR-MPLS provides simplified forwarding without per-flow state
- Label stacking enables traffic engineering and L3VPN services across all sites
- Ring topology provides path redundancy with automatic failover

### 1.3.2 Sample HTTP API Outputs from JCNR3 Cluster

The following examples show real HTTP API outputs from the JCNR3 cluster (jcnr3.demolab) in the topology above, demonstrating the type of operational data available through YAMS:

**Next-hop Table via HTTP API**

**Text Input**: "Get next-hop table from HTTP API in JCNR3 cluster"

**Formatted Output**:

```
================================================================================================================================

NEXT-HOP TABLE (HTTP API) - JCNR3 Cluster
================================================================================================================================

ID Type RefCnt Valid Policy Interface VxLAN
--------------------------------------------------------------------------------------------------------------------------------

1 discard 3 Yes No N/A No
3 l2-receive 7 Yes No N/A No
10 receive 7 Yes No lo No
31 receive 2 Yes No enp10s0 No
33 receive 2 Yes No enp7s0 No
34 receive 2 Yes No enp8s0 No
32 receive 2 Yes No enp9s0 No
25 arp 4 Yes Yes enp7s0 No
26 arp 4 Yes Yes enp9s0 No
6 interface 1 Yes No enp10s0 No
7 interface 1 Yes No enp7s0 No
8 interface 1 Yes No enp8s0 No
... and 40 more entries
--------------------------------------------------------------------------------------------------------------------------------

Total Next-hops: 51
================================================================================================================================
```

**VRF Table via HTTP API**

**Text Input:** "Show VRF table information from HTTP API"

**Formatted Output**:

```
================================================================================================================

VRF TABLE (HTTP API) - JCNR3 Cluster
================================================================================================================

Name UC Index L2 Index VxLAN ID RD
----------------------------------------------------------------------------------------------------------------

default-domain:contrail:ip-fabri... 0 0 0 0.0.0.0:0
srmpls 1 1 0 0.0.0.0:1
----------------------------------------------------------------------------------------------------------------

Total VRFs: 2
================================================================================================================
```

**IPv4 Route Table via HTTP API**

**Text Input**: "Display IPv4 routing table from HTTP API in JCNR3"

**Formatted Output**:

```
================================================================================================================================

IPv4 ROUTE TABLE (HTTP API) - JCNR3 Cluster
================================================================================================================================

Prefix Len VRF Next-hop Label Peer
--------------------------------------------------------------------------------------------------------------------------------

2.2.2.2/32 32 default-domain:contrai... 0 -1 gRPCPeer
3.3.3.3/32 32 default-domain:contrai... 10 -1 gRPCPeer
4.4.4.4/32 32 default-domain:contrai... 35 -1 gRPCPeer
6.6.6.6/32 32 default-domain:contrai... 0 -1 gRPCPeer
192.168.133.0/24 24 default-domain:contrai... 15 -1 gRPCPeer
192.168.133.2/32 32 default-domain:contrai... 25 -1 Local
192.168.133.3/32 32 default-domain:contrai... 33 -1 gRPCPeer
192.168.144.0/24 24 default-domain:contrai... 16 -1 gRPCPeer
192.168.144.3/32 32 default-domain:contrai... 34 -1 gRPCPeer
192.168.155.0/24 24 default-domain:contrai... 20 -1 gRPCPeer
192.168.155.3/32 32 default-domain:contrai... 32 -1 gRPCPeer
192.168.155.6/32 32 default-domain:contrai... 26 -1 Local
192.168.190.0/24 24 default-domain:contrai... 0 -1 gRPCPeer
192.168.200.0/24 24 default-domain:contrai... 21 -1 gRPCPeer
192.168.200.3/32 32 default-domain:contrai... 31 -1 gRPCPeer
... and 5 more routes
--------------------------------------------------------------------------------------------------------------------------------

Total IPv4 Routes: 18
================================================================================================================================
```

**Interface Statistics via HTTP API**

**Text Input**: “Get interface statistics from HTTP API”

**Formatted Output**:

```
================================================================================================================================

INTERFACE STATISTICS (HTTP API) - JCNR3 Cluster
================================================================================================================================

Name Type State IPv4 Address RX Packets TX Packets RX Bytes TX Bytes
--------------------------------------------------------------------------------------------------------------------------------

enp7s0 Physical UP 192.168.133.3 82137 187224 6578982 14312013
enp8s0 Physical UP 192.168.144.3 15234 25891 1423567 2234891
enp9s0 Physical UP 192.168.155.6 455760 50187 39022182 4516830
enp10s0 Physical UP 192.168.200.3 125487 89234 12456789 8923456
lo Loopback UP 3.3.3.3 97670 97670 7813600 7813600
vif0/0 Agent UP N/A 137644 22284 11837384 1961262
--------------------------------------------------------------------------------------------------------------------------------

Total Interfaces: 6 (4 Physical, 1 Loopback, 1 Agent)
================================================================================================================================
```

**HTTP API Analysis**:

- Network Topology Correlation: The interface IP addresses (192.168.133.3, 192.168.144.3, 192.168.155.6, 192.168.200.3) match the ring topology links shown above
- Route Distribution: Routes for all ring segments (133.x, 144.x, 155.x, 200.x networks) are present, confirming BGP/OSPF operation
- Next-hop Analysis: ARP and interface next-hops for physical links (enp7s0-enp10s0) align with the four-port ring configuration
- Traffic Patterns: Interface statistics show active traffic across all ring links, with enp9s0 showing the highest utilization

## 1.4 Comprehensive Use Case: Multi-Cluster JCNR Network Analysis

This use case demonstrates how YAMS enables comprehensive network analysis across multiple JCNR clusters through unified operations. A network operations team needs to perform a complete analysis of their JCNR infrastructure spanning the 4 interconnected clusters shown in the topology above.

### 1.4.1 Scenario: Production Network Health Assessment

Environment:

- 4 JCNR clusters: JCNR3 (SR Control), JCNR4 (SR Control), JCNR6 (SR Transit), JCNR2 (SR Endpoint)
- Each cluster is running OSPF with SR extensions and SR-MPLS L3VPN services in ring topology
- Requirements: Complete SR protocol status, interface statistics, datapath analysis, and specific route investigation

### 1.4.2 Step 1: BGP Summary Across All Clusters

**Objective**: Get BGP neighbor status and session information from all cRPD instances.

**Text Input**: “Show me the BGP summary and neighbor status across all JCNR clusters”

**Sample Output**:

```
 Cluster: JCNR3 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Groups: 2 Peers: 4 Down peers: 0
Table Tot Paths Act Paths Suppressed History Damp State Pending
inet.0 24 12 0 0 0 0
bgp.l3vpn.0 45 23 0 0 0 0

Peer AS InPkt OutPkt OutQ Flaps Last Up/Dwn State
4.4.4.4 64512 12453 12389 0 2 5d 2h 15m Estab
192.168.144.6 64512 12445 12391 0 1 5d 2h 12m Estab
192.168.133.2 64512 9834 9821 0 0 3d 4h 23m Estab

 Cluster: JCNR4 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Groups: 1 Peers: 2 Down peers: 0
Peer AS InPkt OutPkt OutQ Flaps Last Up/Dwn State
3.3.3.3 64512 15234 15198 0 1 6d 1h 45m Estab
192.168.200.2 64512 15229 15195 0 0 6d 1h 42m Estab

 Cluster: JCNR6 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Groups: 1 Peers: 2 Down peers: 0
Peer AS InPkt OutPkt OutQ Flaps Last Up/Dwn State
192.168.144.3 64512 18456 18423 0 0 7d 3h 22m Estab
192.168.155.4 64512 18451 18419 0 1 7d 3h 19m Estab

[JCNR2 cluster output...]
```

**Analysis Results**:

- **JCNR3**: SR-MPLS control node with L3VPN routes, 45 VPN routes distributed
- **JCNR4**: SR-MPLS control node with established sessions, stable label distribution
- **JCNR6**: SR-MPLS transit node functioning correctly with forwarding tables
- **All clusters**: Stable SR-MPLS infrastructure with proper label distribution

### 1.4.3 Step 2: OSPF Summary Across All Clusters

**Objective**: Verify OSPF adjacencies, SR prefix SID distribution, and area information across all clusters.

**Text Input**: “Check OSPF neighbor adjacencies and database status in all clusters”

**Sample Output**:

```
 Cluster: JCNR3 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Address Interface State ID Pri Dead
192.168.133.2 enp7s0.0 Full 2.2.2.2 128 39
192.168.144.6 enp8s0.0 Full 6.6.6.6 128 37

 Cluster: JCNR6 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Address Interface State ID Pri Dead
192.168.144.3 enp7s0.0 Full 3.3.3.3 128 35
192.168.155.4 enp8s0.0 Full 4.4.4.4 128 33

OSPF database, Area 0.0.0.0:
 Type ID Adv Rtr Seq Age Opt Cksum Len
Router 2.2.2.2 2.2.2.2 0x8000002a 156 0x22 0x8c45 48
Router 3.3.3.3 3.3.3.3 0x8000002b 157 0x22 0x7d52 60
Router 4.4.4.4 4.4.4.4 0x8000002c 158 0x22 0x6f61 48
Router 6.6.6.6 6.6.6.6 0x8000002d 159 0x22 0x5e73 60
```

**Analysis Results**:

- **OSPF Adjacencies**: All neighbors in Full state across clusters with SR capability
- **LSA Database**: Consistent topology information with SR prefix SID advertisements
- **SR-MPLS**: Area 0.0.0.0 stable with 4 SR-enabled routers, prefix SID distribution working

### 1.4.4 Step 3: Interface Statistics from All JCNR Clusters

**Objective**: Collect comprehensive interface statistics from DPDK data plane.

**Text Input**: “Get interface statistics and packet counters from the DPDK data plane across all clusters”

**Sample Output**:

```
 Cluster: JCNR6 | Pod: vrdpdk-abc123 | Type: DPDK
================================================================================
vif0/1 PCI: 0000:07:00.0 NH: 6 MTU: 9000
          Type:Physical HWaddr:52:54:00:00:a9:14 IPaddr:192.168.155.6
          Vrf:0 Mcast Vrf:0 Flags:L3Vof QOS:0 Ref:12
          RX packets:22522950482 bytes:1531390015403 errors:0
          TX packets:1778842 bytes:134599516 errors:0
          Drops:28977242

vif0/2 PCI: 0000:08:00.0 NH: 7 MTU: 9000
          Type:Physical HWaddr:52:54:00:99:5c:41 IPaddr:192.168.144.6
          Vrf:0 Mcast Vrf:0 Flags:L3Vof QOS:0 Ref:7
          RX packets:453749 bytes:40234716 errors:0
          TX packets:131647 bytes:11789166 errors:0
          Drops:0

 Cluster: JCNR3 | Pod: vrdpdk-def456 | Type: DPDK
================================================================================
vif0/1 PCI: 0000:07:00.0 NH: 8 MTU: 9000
          Type:Physical HWaddr:52:54:00:11:b2:33 IPaddr:192.168.133.3
          Vrf:0 Mcast Vrf:0 Flags:L3Vof QOS:0 Ref:9
          RX packets:18334567891 bytes:1245678912345 errors:0
          TX packets:2456789 bytes:198765432 errors:0
          Drops:15678

[JCNR4 and JCNR2 interface statistics...]
```

**Analysis Results**:

- **Traffic Volume**: Heavy traffic on primary interfaces (22B+ packets)
- **Error Rates**: Zero errors on most interfaces, indicating healthy data plane
- **Drop Analysis**: Some drops detected on high-traffic interfaces (normal behavior)

#### 1.4.5 Step 4: JCNR Summary from All Clusters

**Objective**: Complete datapath and control plane analysis, combining all JCNR components.

**Text Input**: "Provide a comprehensive JCNR summary with datapath, control plane, and HTTP API data from all clusters"

**Sample Output**:

```
JCNR Datapath Summary with Route/Next-hop/Flow Analysis + HTTP API + Junos CLI
================================================================================
 Cluster: JCNR3 | Pod: vrdpdk-abc123 | Type: DPDK
================================================================================
Next-hops: 47 total (receive, interface, ARP, tunnel types)
Interfaces: 5 physical interfaces with 18B+ packets processed
Routes: 4,603 IPv4 routes, 12,507 IPv6 routes in forwarding table
Flows: 2,491 flows created, active TCP/UDP sessions
MPLS: 11 labels configured (transport and VPN labels)

HTTP API Summary:
   - VRFs: 2 (default + srmpls VPN)
   - IPv4 Routes: 18 active routes with next-hop mapping
   - IPv6 Routes: 8 routes including link-local addresses

 Cluster: JCNR3 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Router ID: 3.3.3.3
Route Tables: inet.0 (19 destinations), bgp.l3vpn.0 (45 VPN routes)
Protocols: OSPF active (2 neighbors), BGP active (3 peers including multi-hop to JCNR4)
FIB: 21 routes installed for forwarding

 Cluster: JCNR6 | Pod: vrdpdk-ghi789 | Type: DPDK
================================================================================
Next-hops: 52 total (transit role with more tunnel next-hops)
Interfaces: 4 physical interfaces with 22B+ packets processed
Routes: 5,127 IPv4 routes, 13,245 IPv6 routes in forwarding table
Flows: 3,156 flows created, high transit traffic
MPLS: 23 labels configured (extensive transit labeling)

 Cluster: JCNR6 | Pod: jcnr-0-crpd-0 | Type: cRPD
================================================================================
Router ID: 6.6.6.6
Route Tables: inet.0 (22 destinations), bgp.l3vpn.0 (38 VPN routes)
Protocols: OSPF active (2 neighbors), BGP active (2 peers)
FIB: 24 routes installed for forwarding

[Similar summaries for JCNR4 and JCNR2...]
```

**Analysis Results**:

- **Datapath Health**: All clusters showing healthy SR-MPLS packet forwarding
- **Route Scale**: Consistent route counts with proper SR label distribution
- **Protocol Status**: OSPF stable with SR prefix SID distribution across all clusters
- **VPN Services**: SR-MPLS L3VPN routes are properly distributed with label stacking

#### 1.4.6 Step 5: Specific Route Analysis

**Objective**: Investigate specific route 30.30.24.11/32 in detail from JCNR3 cluster.

**Text Input**: "Analyze the route 30.30.24.11/32 in JCNR3 cluster, showing both control plane and data plane details"

**Route Analysis Output**:

Control Plane (cRPD):

```
30.30.24.11/32 (1 entry, 1 announced)
      *BGP Preference: 170/-101
            Route Distinguisher: 64512:1
            Next hop type: Indirect, Next hop index: 0
            Address: 0x55c8e2f4a8c0
            Next-hop reference count: 4
            Source: 4.4.4.4
            Protocol next hop: 4.4.4.4
            Indirect next hop: 0x2 no-forward INH Session ID: 0x0
            State: <Active Int Ext>
            Local AS: 64512 Peer AS: 64512
            Age: 2d 4:08:38
            Validation State: unverified
            Task: BGP_64512.4.4.4.4+179
            AS path: I
            Communities: target:64512:1
            Import Accepted
            VPN Label: 59
            Localpref: 100
            Router ID: 4.4.4.4
            Primary Routing Table: bgp.l3vpn.0
            Indirect next hops: 1
                  Protocol next hop: 4.4.4.4 Metric: 2
                  Indirect next hop: 0x2 no-forward INH Session ID: 0x0
                  Indirect path forwarding next hops: 2
                        Next hop type: Router
                        Next hop: 192.168.133.2 via enp7s0.0
                        Next hop: 192.168.155.6 via enp9s0.0
                  4.4.4.4/32 Originating RIB: inet.3
                    Metric: 2 Node path count: 1
                    Forwarding nexthops: 2
                        Nexthop: 192.168.133.2 via enp7s0.0
                        Nexthop: 192.168.155.6 via enp9s0.0, Push 14400
```

Data Plane (DPDK):

```
Match 30.30.24.11/32 in vRouter inet4 table 0/1/unicast
Destination PPL Flags Label Nexthop Stitched MAC(Index)
30.30.24.11/32 0 PT - 54 -

Next-hop 54: Indirect > Composite ECMP (NH 50)
     NH 39: MPLS Tunnel via enp7s0 (192.168.133.2)
        Labels: VPN=59, Transport=14400
        MAC: 52:54:00:ee:73:9b > 52:54:00:fe:c1:b8
     NH 40: MPLS Tunnel via enp9s0 (active path)
        Labels: VPN=59, Transport=14400
        MAC: 52:54:00:00:a9:14 > 52:54:00:a0:7c:7f
        Hit Count: 22,467,245,874 packets
```

**Route Analysis Summary**:

- **Route Type**: BGP L3VPN route (RD: 64512:1)
- **Source**: PE router 4.4.4.4 via iBGP
- **MPLS Labels**: VPN label 59, Transport label 14400
- **Load Balancing**: ECMP across 2 paths (enp7s0, enp9s0)
- **Active Forwarding**: 22+ billion packets via enp9s0 path
- **State**: Route active in both control and data planes

### 1.4.7 Use Case Summary

This comprehensive analysis demonstrates YAMS's capability to:

- **Unified Protocol Analysis**: BGP and OSPF status across all clusters in single commands
- **Performance Monitoring**: Interface statistics and traffic analysis from DPDK data plane

- **Complete Infrastructure View**: Combined control plane and data plane visibility
- **Detailed Route Investigation**: Deep-dive analysis of specific routes with complete forwarding details

Traditional Approach: Manual cluster-by-cluster analysis with individual tool execution. YAMS Approach: Unified automated analysis across all clusters simultaneously.

**Key Benefits Demonstrated**:

- **Significant time reduction** for multi-cluster analysis workflows
- **Unified visibility** across JCNR's three-tier architecture
- **Consistent data format** regardless of cluster location or deployment method
- **Deep diagnostic capabilities** for specific route troubleshooting and analysis

## 1.5 Installation and Setup

### 1.5.1 Prerequisites

- Docker and Docker Compose installed
- Kubernetes cluster access with appropriate RBAC permissions
- SSH access to remote clusters (if applicable)
- VS Code with MCP support (optional but recommended)

### 1.5.2 Quick Start with Docker

```
# Clone YAMS repository
git clone https://github.com/Juniper/yams.git yams-github
cd yams-github

# Quick start with Docker
./docker-run.sh

# Verify installation
curl http://localhost:40041/health
```

### 1.5.3 Cluster Configuration

Create **clusters/clusters.json** with your JCNR cluster details. Here's the actual configuration from the YAMS workspace showing different cluster connection methods:

```
{
  "jcnr3-cluster": {
    "kubeconfig_path": "/app/kubeconfigs/jcnr3-kubeconfig",
    "description": "JCNR3 Kubernetes cluster",
    "jcnr_command_list": "/app/clusters/jcnr-command-list.json",
    "pod_command_list": "/app/clusters/pod-command-list.json"
  },
  "jcnr4-cluster": {
    "kubeconfig_path": "/home/jcnr4/.kube/config",
    "ssh": {
      "host": "jcnr4.demolab",
      "username": "jcnr4",
      "key_path": "/app/.ssh/jcnr4",
      "k8s_host": "jcnr4.demolab",
      "k8s_port": 6443,
      "local_port": 16444
    },
    "jcnr_command_list": "/app/clusters/jcnr-command-list.json",
    "pod_command_list": "/app/clusters/pod-command-list.json"
```

```
  },
  "jcnr2-cluster": {
    "kubeconfig_path": "/home/jcnr2/.kube/config",
    "ssh": {
      "host": "jcnr2.demolab",
      "username": "jcnr2",
      "password": "<password>",
      "k8s_host": "jcnr2.demolab",
      "k8s_port": 6443,
      "local_port": 16443
    },
    "jcnr_command_list": "/app/clusters/jcnr-command-list.json",
    "pod_command_list": "/app/clusters/pod-command-list.json"
  },
  "jcnr6-cluster": {
    "kubeconfig_path": "/home/jcnr6/.kube/config",
    "ssh": {
      "host": "jcnr6.demolab",
      "username": "jcnr6",
      "key_path": "/app/.ssh/jcnr6",
      "k8s_host": "jcnr6.demolab",
      "k8s_port": 6443,
      "local_port": 16445
    },
    "jcnr_command_list": "/app/clusters/jcnr-command-list.json",
    "pod_command_list": "/app/clusters/pod-command-list.json"
  }
}
```

**Configuration Examples Explained**:

- **jcnr3-cluster**: Direct kubeconfig access for local/accessible clusters
- **jcnr4-cluster**: SSH tunnel with key-based authentication to jcnr4.demolab
- **jcnr2-cluster**: SSH tunnel with password authentication to jcnr2.demolab
- **jcnr6-cluster**: SSH tunnel with key-based authentication to jcnr6.demolab

**Key Configuration Parameters**:

- **kubeconfig_path**: Path to the Kubernetes configuration file
- **description**: Human-readable cluster description
- **ssh.host**: SSH jump host for remote clusters
- **ssh.username**: SSH username for authentication
- **ssh.key_path**: Path to SSH private key (alternative to password)
- **ssh.password**: SSH password (alternative to key-based auth)
- **ssh.k8s_host**: Kubernetes API server hostname/IP
- **ssh.k8s_port**: Kubernetes API server port (default: 6443)
- **ssh.local_port**: Local port for SSH tunnel forwarding
- **jcnr_command_list**: Path to JCNR-specific command configuration
- **pod_command_list**: Path to general pod diagnostic commands

### 1.5.4 Command Configuration

Configure JCNR-specific command sets in **jcnr-command-list.json** (actual configuration from YAMS workspace):

```
{
  "datapath_commands": [
    "nh --list",
    "vif --list",
    "rt --dump 0",
    "rt --dump 0 --family inet6",
    "frr --dump",
```

```
    "mpls --dump",
    "flow -l"
  ],
  "junos_cli_commands": [
    "show interfaces extensive",
    "show route summary",
    "show route",
    "show route protocol bgp",
    "show route protocol direct",
    "show route protocol static",
    "show bgp summary",
    "show bgp neighbor",
    "show protocols bgp",
    "show configuration"
  ],
  "http_endpoints": {
    "nh_list": "Snh_NhListReq?type=&nh_index=&policy_enabled=",
    "vrf_list": "Snh_VrfListReq?name=",
    "inet4_routes": "Snh_Inet4UcRouteReq?x=0",
    "inet6_routes": "Snh_Inet6UcRouteReq?x=0",
    "interface_list": "Snh_ItfReq?name=",
    "flow_list": "Snh_FetchFlowRecord?x=0"
  },
  "http_port": 8085,
  "analysis_config": {
    "max_display_lines": 50,
    "max_http_display_chars": 5000,
    "enable_detailed_analysis": true,
    "truncate_large_outputs": true
  }
}
```

Pod Commands (**pod-command-list.json** - actual configuration):

```
{
  "description": "Standard diagnostic commands for pod health checking and system information
      gathering",
  "commands": [
    "hostname",
    "uptime",
    "ps",
    "df -h",
    "free -m",
    "cat /proc/meminfo",
    "cat /proc/cpuinfo",
    "ls -la /tmp"
  ]
}
```

**Configuration Sections Explained**:

- datapath_commands: DPDK vRouter commands executed in vrdpdk pods
- junos_cli_commands: Junos CLI commands executed in cRPD pods (automatically prefixed with cli -c)
- http_endpoints: Sandesh HTTP API endpoints for real-time data access
- http_port: Port number for Contrail agent HTTP API (default: 8085)
- analysis_config: Output formatting and analysis parameters
  - max_display_lines: Limit displayed lines for large outputs
  - max_http_display_chars: Character limit for HTTP API responses
  - enable_detailed_analysis: Enable comprehensive analysis features
  - truncate_large_outputs: Automatically truncate large command outputs

**Command Customization**:

- Commands are executed exactly as specified in the JSON configuration
- DPDK commands target **/contrail** namespace vrdpdk pods
- Junos CLI commands target **/jcnr** namespace cRPD pods
- HTTP endpoints are accessed via the configured port on agent pod IPs

- Custom command sets can be configured per cluster for different environments

**Note**: Both JCNR and pod command lists can be modified to include desired commands based on your specific operational requirements. Refer to the JCNR documentation for comprehensive command configuration options and best practices for customizing command sets for different deployment scenarios.

#### 1.5.5 VS Code Integration

Add to your VS Code **settings.json** for Copilot Chat integration:

```
{
  "mcp": {
    "servers": {
      "yams-mcp-server": {
        "url": "http://127.0.0.1:40041/mcp/"
      }
    }
  }
}
```

### 1.6 Benefits and Impact

#### 1.6.1 Operational Benefits

- **Reduction in analysis time** in multi-cluster analysis time
- **Unified interface** for JCNR's three-tier architecture
- **Consistent operations** across on-premises and cloud deployments
- **Enhanced troubleshooting** with correlated data from all components
- **Reduced human error** through automated command execution
- **Improved visibility** into JCNR datapath and control plane status

#### 1.6.2 Technical Advantages

- **Real-time data access** via Sandesh HTTP API integration
- **Flexible configuration** with external JSON command sets
- **Secure access** through SSH tunneling and key-based authentication
- **Scalable architecture** supporting 50+ clusters
- **Modern integration** with VS Code and AI-powered workflows

### 1.7 Getting Started

1. **Download and install YAM**S using the Docker quick-start method
2. **Configure your JCNR clusters** in the JSON configuration format
3. **Set up command lists** for your specific JCNR deployment patterns
4. **Integrate with VS Code** for enhanced workflow capabilities
5. **Begin unified JCNR management** across all your clusters

YAMS provides the foundation for modern, scalable JCNR operations where multi-cluster complexity meets unified simplicity.

## 1.8 Conclusion

YAMS transforms JCNR management from a manual, cluster-by-cluster process into a unified, automated workflow. By providing specialized tools for JCNR's three-tier architecture and enabling comprehensive multi-cluster analysis, YAMS addresses the key operational challenges facing network teams managing cloud-native routing infrastructure.

The combination of DPDK data plane tools, cRPD control plane integration, and Contrail agent management provides complete visibility into JCNR deployments. With features like automated BGP/OSPF analysis, interface statistics collection, comprehensive JCNR summaries, and detailed route investigation, YAMS enables network engineers to operate at scale while maintaining deep technical insight.

For organizations running JCNR in production, YAMS offers a practical solution that reduces operational complexity, improves troubleshooting efficiency, and enables modern DevOps workflows for network infrastructure management.

YAMS provides the foundation for modern, scalable JCNR operations where multi-cluster complexity meets unified simplicity.

## 1.9 Useful links

- JCNR Landing Page: https://www.juniper.net/documentation/product/us/en/juniper-cloud-native-router/
- JCNR Troubleshooting guide https://www.juniper.net/documentation/us/en/software/cloud-native-router25.2/cloud-native-router-deployment-guide/topics/topic-map/jcnr-troubleshoot-deployment.html
- JCNR tool list: https://www.juniper.net/documentation/us/en/software/contrail-networking20/contrail-analytics-troubleshooting-guide/topics/concept/contrail-tools.html
- JCNR Deployment Guide: https://www.juniper.net/documentation/us/en/software/cloud-native-router25.2/cloud-native-router-deployment-guide/index.html
- YAMS README: https://github.com/Juniper/yams/blob/main/README.md
- JCNR Agent HTTP API Introspect Guide: https://www.juniper.net/documentation/us/en/software/cloud-native-router24.2/cloud-native-router-user/topics/concept/jcnr-introspect.html
- cSRX Landing Page: https://www.juniper.net/us/en/products/security/srx-series/csrx-containerized-firewall.html

## 1.10 Glossary

- cRPD: Containerized JUNOS Routing Protocol Daemon
- cSRX: Containerized JUNOS SRX
- K8S: Kubernetes
- JCNR: Juniper Cloud Native Router
- MCP: Model Context Protocol
- SR-MPLS: Segment Routing MPLS
- YAMS: Yet Another MCP Server

## 1.11 Acknowledgments

The author would like to thank Vinod Nair and JCNR team for reviewing and trying this MCP server.




DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

**HEWLETT PACKARD ENTERPRISE**

**HPE.com**




